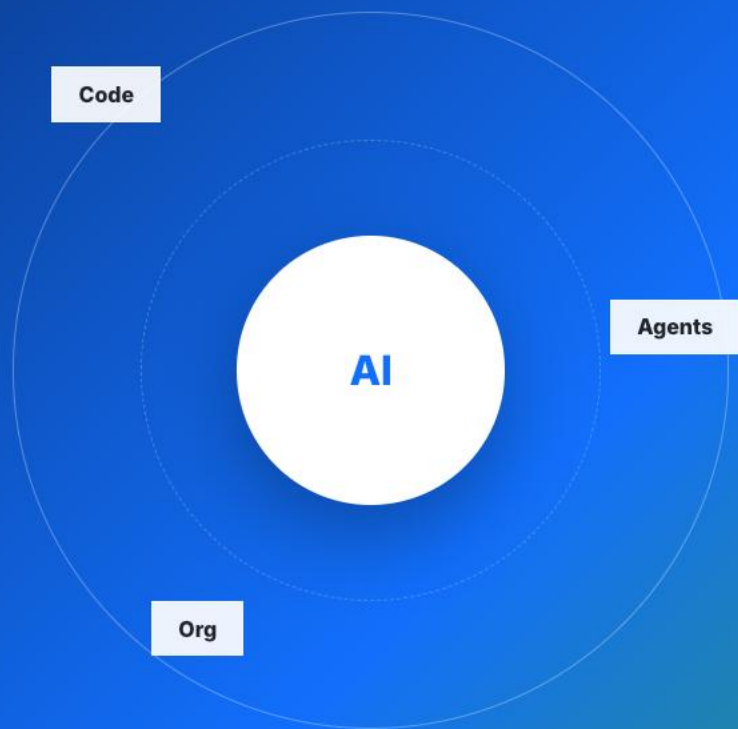


Code, Agents and Parallel Worlds

AI 研发下半场的组织进化 与
Agent Centric Engineering 实践

董鑫 · 华为2012 编译器与编程语言实验室 项目群总监



AI ADOPTION LAYERS

世界已经形成四大AI 圈层

AI 能力不是平均扩散，而是沿着工具深度、付费壁垒、Agent 编排能力形成陡峭分层。

84%**被遗忘的群体**

全球化数字鸿沟仍在，广大欠发达地区和人群尚未有效利用 AI 核心能力。

16%**浅表应用**

多数用户停留在搜索、基础写作等日常任务，未触及生产力重塑。

0.3%**付费壁垒**

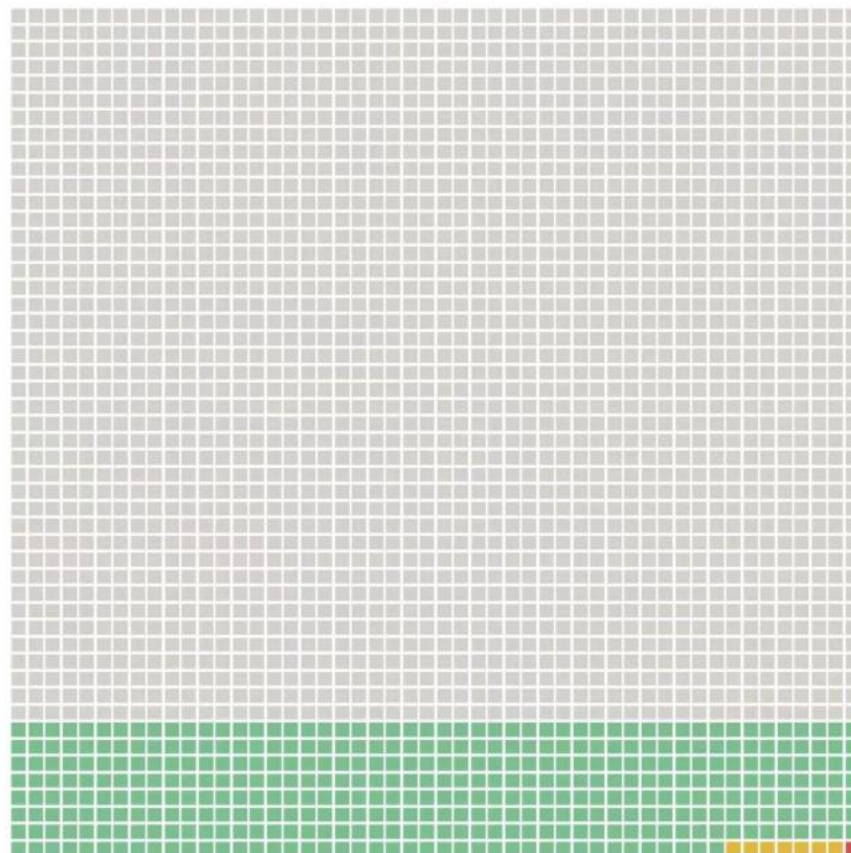
高阶模型、Prompt 技巧与 workflow 意识，让精英用户率先突破效率边界。

0.04%**规则制定者**

利用 Agent、多模型协作和全栈能力，重构行业工作流的少数人。

Each dot is ~3.2 million people

2,500 dots = 8.1 billion humans. Color = most advanced AI interaction, Feb 2026.



Never used AI · ~6.8B (84%) Free chatbot user · ~1.3B (16%)
Pays \$20/mo for AI · ~15-25M (~0.3%) Uses coding scaffold · ~2-5M (~0.04%)

FIVE-STEP TRANSFORMATION

仓颉团队 AI 驱动开发五步走

从“绿”到“红”的转变，不是工具采用曲线，而是研发组织能力被持续拉升的过程。



EFFICIENCY PROOF

个人级任务效率提升数据

实践数据说明：AI 的收益横跨三方库、编译器、API、资料、UI 与工具链，不是单点试验。

任务	团队	效率提升
三方库开发效率中的实践	三方库团队	产能提升 2.7X-4.5X 1.1 Kloc/人月 → 3-5 Kloc/人月
编译器 Syntax 库适配	编译器团队	60% 5 人天 → 2 人天
编译器 CHIR 插件实现	编译器团队	25% 20 人天 → 15 人天
编译器性能劣化点引入分析	编译器团队	75% 4 人时 → 1 人时
AI 提升工具开发效率实践	工具团队	95% 10 人天 → 0.5 人天
API 测试用 AI 例生成中的实践	API 团队	80% 5 人天 → 1 人天
AI 在 C++ Crash 类稳定性问题分析定位中的经验分享	API 团队	92% 3 人天 → 0.25 人天
AI 辅助编译器前端代码开发	编译器团队	40% 5 人天 → 3 人天
使用 AI 辅助开发者文档英文文化的实践	API + 资料团队	90% 40 人天 → 4 人天
AI 在资料整改中的效率提升实践	API 团队	28% 54 人天 → 39 人天
AI 在 Cangjie UI 1.2 项目中的效率提升实践	鸿蒙 UI 团队	33%-80% 3-5 人天 → 1-2 人天
Rules+MCP 工作流对团队新人上手效率的提升	鸿蒙 UI 团队	33%-80% 3-5 人天 → 1-2 人天
Rules+MCP 对团队新人首次独立提交时间的加速	鸿蒙 UI 团队	50% 5 人天 → 2.5 人天
Rule+MCP 对 Code Review 效率的提升实践	鸿蒙 UI 团队	60%-100% 5-8 次/PR → 0-2 次/PR
使用 AI 开发代码仓质量看护看板系统的实践	工具团队	80%-86% 10-14 人天 → 2 人天

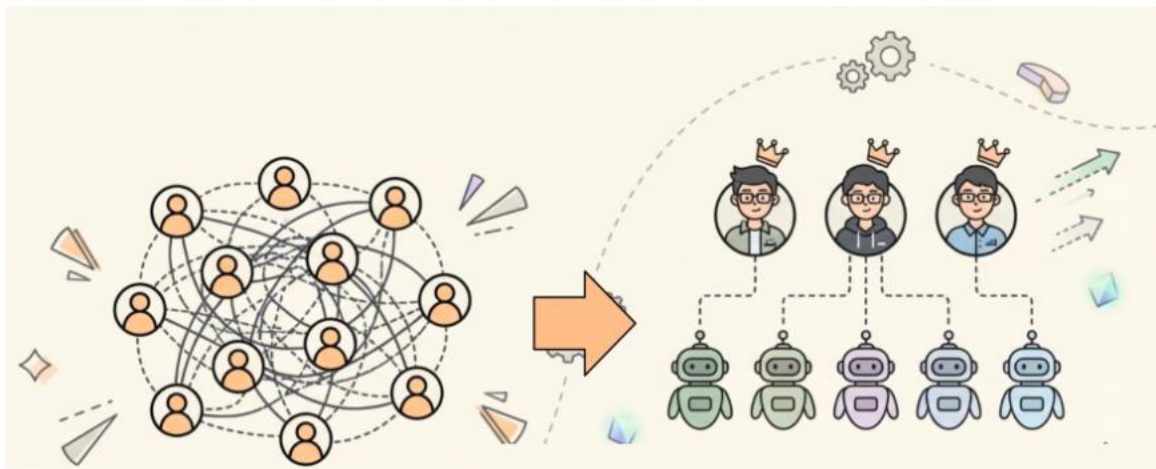
CODING IS ONLY THE TIP

编码只占 1/5-1/3，研发 E2E 才是主战场

大型项目中真正的研发 E2E 流程包含需求分析、对齐、实现、验证、修复等多个环节，AI 的提效空间远不止于代码生成。

现实

理想



1/5-1/3

编码开发时间只占项目一小部分

大型项目中真正写代码的时间只占 1/5-1/3，其余都是非编码工作。

沟通

大量时间花在沟通对齐上

需求澄清、方案对齐、跨团队协作、评审反馈，占据工程师大部分日常。

E2E

真正的研发 E2E 流程

需求分析·对齐·实现·验证·修复——端到端每个环节都决定交付速度。

AI 提效的真正杠杆，是把 E2E 全流程都纳入治理范围

BEYOND CODE GENERATION

Agentic 时代不是降低人的要求，而是抬高团队的组织能力

团队从“碳基人员集合”，演进为“碳基 + 硅基”的连续作战系统：人定义目标、边界和质量，Agent 承担标准化、可验证、可持续的执行。

01

CORE THESIS

01

人的角色上移

从编码执行者，转向目标设定、架构判断、质量仲裁、异常处置。

编码执行



目标 / 架构 / 质量

02

Agent 最大化自主执行

最大化保障实验探索、需求分析、代码开发、结果验证的持续运转，即最大化创新的催化和需求的交付。

单点提效



持续运转

03

组织动态成军

人不再被固定职能锁死，而是以需求为核心的“液态组织”。

固定职能



液态组织

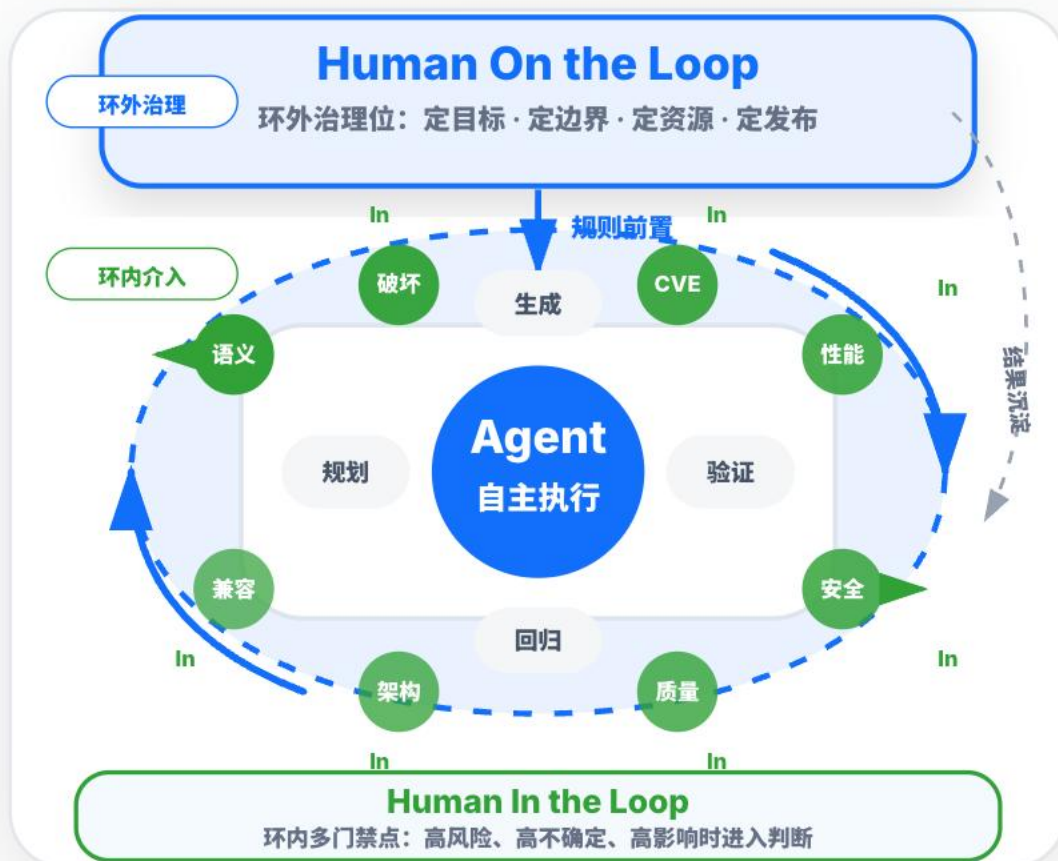
HUMAN IO LOOP

HUMAN IO LOOP

Human On the Loop + Human In the Loop

IO 不是输入输出的简单缩写，而是两种人的介入方式：人在环上做治理，人在环中做关键节点判断。

核心目标：让 Agent 最大化自主执行，同时把人的注意力集中在策略、边界、质量、风险和学习沉淀上。



DIFFERENCE AND UNITY · LAYERED SYSTEM

Agent Centric Engineering 理念

统一方式不是增加人审，而是把治理规则、触发条件、验证资产沉淀进 Agentic Centric Engineering。

SYSTEM CONTROL

CONTINUOUS AGENT LOOP

RESULT TRUST

Human On the Loop

系统可控



可控：定规则、定边界、定节奏。



Human In the Loop

结果可信



可信：做判断、做仲裁、做背书。

规则前置

Agent 执行

风险触发

人审裁决

资产沉淀

AGENT CENTRIC ENGINEERING

从 ACE 理念到 ACE Harness 开源平台

Agent为核心，解放Agent生产力

从Codeing Agent单兵作战，到全流程Agent团队

工程师：导演 + 编剧

编排工作流程，把控质量进度，纠正偏差；明确需求，拆解任务，设定边界。



Agent：舞台中心

承担核心开发、辅助生成、批量测试、日志分析和可验证执行。

主角

核心功能开发与候选方案生成。

配角

辅助代码编写、文档、用例补齐。

群演

批量测试、日志分析、回归验证。

代码开发只是项目的 1/5-1/3；ACE 的关键，是让分析、实现、验证、修复可以连续运转。

ACE Harness 产品总览



STANDARD LIBRARY COMMUNITY WORKFLOW

案例：社区Issue7x24自动分析 + Bug自动修复

社区 Issue 自动分析

5 Agent

角色分工，状态机按步骤流转

Supervisor

构建任务环路、拆解死循环

Auto 24x7

凌晨自动运行，人休息时为次日准备输入

50%+

问题分发与解决效率提升



每天凌晨自动拉取社区新增 issue，由五类角色 Agent 按状态机流转完成提取、清洗、环境准备、问题分析、验证与通知，输出初步分析报告并告知库 Owner，问题分发与解决效率提升 50%+。

Bug 自动修复案例

Extend ICE: 多继承路径成员合并

状态机工作流通过红蓝对抗与人工审查，实现从复现到验证的完整闭环，修复 VTable 构建缺陷。

总耗时
~1h改动规模
+1 / -6 行回归结果
38/38状态转移
9 / 30综合评分
9.7 / 10

已合入主分支 ✓

核心 7 状态 (含回退)

STATE 01
复现确认STATE 02
根因分析STATE 03
方案设计STATE 04
人工审查STATE 05
代码修复STATE 06
回归验证修复: VTableGenerator.cpp:454-460
缺少复合泛型递归替换

代码修复 → 人工审查回退 1 次

红蓝对抗与验证

Defender 实现

Attacker 挑战

Judge 裁决

ICE 消除 8/8

压力 24/24

CASE 01 · HIGH COUPLING

In & On: 高耦合仓颌 - 编译器内存优化

高耦合编译器场景里，人的价值不是逐步驱动 AI，而是把目标、接口边界、验证环境一次性前置，再在方案和结果两个门禁点审查。

30%
编译器内存降低

1月→2周
预估开发周期压缩

2道
方案与结果门禁

旧流程卡点：注意力被连续打断

- Issue 分析、方案设计、实现、自验证都需要开发者主驱动 AI。
- 编译器改动必须同步考虑兼容性、性能、IDE、调试接口影响。
- 需求到 PR 缺少分阶段验证和自修复，人工反复补输入、改方向。

Human On：把治理条件前置

- 目标：内存优化；约束：不破坏兼容性、性能、IDE 与调试接口。
- 环境：工具链、Skills、Agent、接口列表、编译器领域知识。
- 节奏：只在方案确认、结果确认、PR 提交触发人介入。

人从“连续推动者”变成“边界与质量仲裁者”

Agent 承担方案设计、代码开发、用例开发、功能验证、性能验证，失败后回到对应步骤自修复。

新工作流：Issue 到 PR 的 Human IO Loop

把人的介入固定在输入治理和关键门禁，让 Agent 在中间段持续执行。

Human On Agent Human In



On the Loop 负责可控

- 定目标、边界、资源、发布动作。
- 把验证资产和接口约束前置。

Agent 负责连续执行

- 分析、设计、实现、测试、性能验证。
- 通过多角色分工完成自研与它验。

In the Loop 负责可信

- 审方案风险、架构影响、兼容性。
- 审最终代码、用例、测试和性能结果。

ACE HARNESS · PRODUCT VIEW

ACE Harness：从任务编排到执行验证

用平台化界面承载 Agent 工作流，让任务拆解、执行过程、结果验证和资产沉淀可观
察、可复用。

workflow · execution · evidence

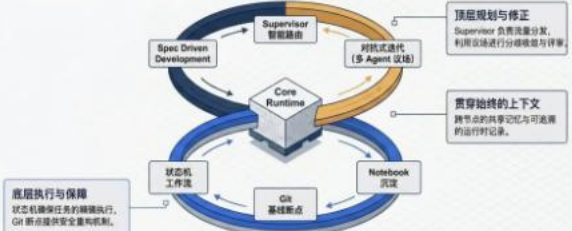
超越单点对话，建立全生命周期智能工程治理

复杂的研发任务无法仅靠大模型的单点问答完成。ACE Harness 提供系统级控制台，将分散的 AI 能力约束在严谨的工程闭环规范之中。



ACE 认知与执行双螺旋架构

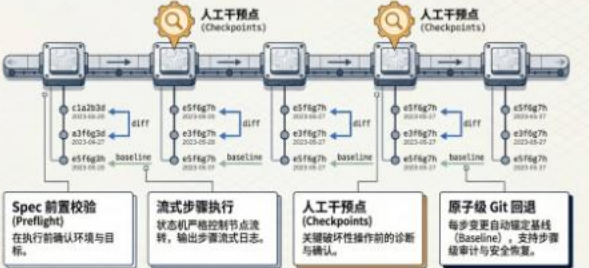
复杂的研发任务无法仅靠大模型的单点问答完成。ACE Harness 提供系统级控制台，将分散的 AI 能力约束在严谨的工程闭环规范之中。



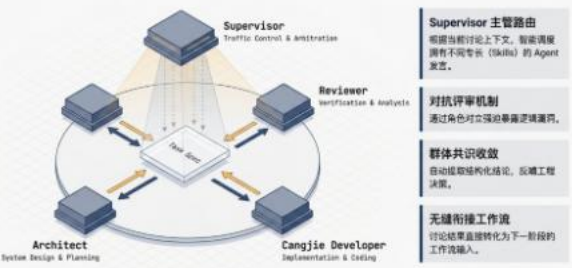
多维协作机制：匹配不同复杂度的工程任务

	基础对话 Standard Chat	状态机工作流 State Machine Workflow	多 Agent 设场 Multi-Agent Arena
适用场景	单点问答、 代码片段生成	确定性工程任务、 代码重构、测试	方案讨论、复杂分歧 收敛、执行复盘
系统机制	大模型基础推理	步骤流式检查点、 Git 变更与对比	Supervisor 流量路由、 角色对抗评审
记忆持久度	单次会话级上下文	工作流生命周期与 工程产物	多层永久群体记忆与 与全局沉淀

状态机驱动的工程装配线与安全回退



设场 (Arena)：对抗式迭代与分歧收敛中心



多维全景驾驶舱：人机协作的操作动线

Multidimensional Panoramic Cockpit: Human-Machine Collaboration Operating Flow



ACE Harness 把多 Agent 协作从“对话窗口”提升为“工程控制台”：过程留痕、结果可审、能力可沉淀。

CASE 02 · LOW COUPLING / HIGH KNOWLEDGE

In & On: 低耦合场景 - 库性能优化

标准库场景耦合较低，但高度依赖集合数据结构知识。Human IO Loop 把专家指导、实习生执行和 Agent 编排组织成可复制流程，减少等待和反复补上下文。

+40%

单生产者单消费者性能

+30%

多生产者单消费者性能

1周

从 1 个月预估压缩

旧流程卡点：公共库人力冲突

- 库团队同时支撑鸿蒙、公开、MetaERP、数通、生态等多个领域。
- 新 issue 需要等待集合数据结构专家空闲，常出现约两周等待。
- AI 缺仓颌库领域知识，开发者要持续补上下文、校验和修正。

Human On：先编组，再执行

- 组队：集合数据结构领域员工指导一名实习生，Agent 承担执行主体。
- 目标：功能正确性、性能提升、兼容性不受影响。
- 环境：编译器、Skills、Agent、库领域知识、基准与验证链路。

把“等专家亲自做”变成“专家设边界，团队带 Agent 做”

经验沉淀到知识库、基准、流程和 Agent Skills 后，同类性能问题可以更快复用。

新 workflow：标准库性能 Issue 的闭环

专家不再被完整执行流程占用，而是在目标定义、方案审查、结果确认中放大判断力。

Human On

Agent

Human In

Human

01

Issue 入池

客户提交性能 issue；DTSE 判断为库问题，移交标准库团队。

Human On

02

目标与编组

专家指导实习生，前置功能、性能、兼容性目标与验证环境。

Agent

03

方案生成

Agent 基于知识库和基准分析问题，输出候选方案并完成自研、它验。

Human In

04

双人审查

实习生和领域专家共同审设计；有风险就要求 Agent 修正并再验证。

Agent

05

实现与基准

Agent 开发功能代码、测试用例，执行功能与性能验证，输出提升结果。

Human In/On

06

交付闭环

实习生确认代码、用例和性能；达标后通知 Agent 提 PR，进入检视与客户验证。

On the Loop 解决匹配

- 把领域专家、实习生、Agent 编成临时攻坚队。
- 把目标、知识、基准、兼容约束一次性前置。

Agent 解决执行

- 读基准、定位瓶颈、生成候选优化方案。
- 补代码、补用例、跑功能和性能验证。

In the Loop 解决可信

- 专家审方案和兼容性，实习生审结果。
- 不符合预期回到实现阶段，形成闭环自修复。

02

ATM

Ability-based Task Matching

ATM 是 Human IO Loop 下的作战阵型：用能力标签把任务池、人才池、Agent 池和优先级队列连接起来，让任务优先匹配最合适的人机组合。

从固定分工，转向基于能力标签的动态编组



液态组织：需求攻坚小队 + Agent 工作流 + 验证门禁

OPTIMAL ALLOCATION · FASTER TTM

CASE 03 · ASCEND OPERATOR MIGRATION

昇腾算子迁移：Agent 中枢编组

面对数百个 Triton 算子迁移、三层技术栈和 A/B/C 客户持续优化需求，ATM 的关键不是固定分工，而是让 Agent 按任务标签调度能力模块。

6人

未增加团队编制

3+

并行外部需求

12.15×

FLA 最高加速比

80%+

平台达标率阶段目标

任务复杂度：不是单点优化

- GPU 平台成熟 Triton 算子生态迁移到昇腾 NPU。
- 跨 AI 模型语义、编译器 IR、NPU 后端三层技术栈。
- A/B/C 客户持续输入精度、性能、亲和接口等外部需求。

旧组织卡点：固定编制跟不上需求节奏

- 需求持续涌入，但岗位和排期固定，借调协调成本高。
- 个人能力偏单一，任务来了常常找不到完整能力组合。
- 分析等人、开发等环境、测试等结果，流程串联造成等待。

ATM 改变的是“能力调度方式”

任务先进入 Agent 中枢，Agent 识别能力缺口，再按需挂载 AI 专家、编译器骨干、工具工程骨干，完成后立即释放。

动态编组结构图：需求池 → Agent 中枢 → 能力模块

Agent 不再等人分配任务，而是持续拉取、匹配、执行、验证、修复；人只在关键节点被挂载。



ONE WEEK · THREE DEMAND LINES

一周三线需求：按任务动态成军

ATM 的价值体现在调度节奏上：不同需求进入同一 Agent 中枢，系统根据能力缺口挂载不同人员，任务结束后人员回池，Agent 继续流水线。

1069 μ s

紧急算子原始延迟

2.557 μ s

优化后延迟结果

0人

批跑统计人工介入

从0

自动积累 Pattern 库

MON

1人挂载

A 客户 FLA 算子精度问题

■ 触发：A 客户提交 FLA 算子优化需求。

■ Agent 判断：识别为 AI 语义与精度风险，需要语义正确性审查。

■ 动态编组：挂载 1 名 AI 专家，在方案设计节点审核优化方向。

■ 继续执行：审核完成后人员释放，Agent 继续生成代码、验证并记录日志。

ATM 机制

能力标签：AI 语义 / 精度 / FLA；人员只在关键门禁介入。

WED

2人攻坚

B 客户紧急延迟 Gap

■ 触发：客户算子延迟 1069 μ s，性能 Gap 大，进入紧急队列。

■ Agent 判断：并行启动多条优化路径，发现需要编译器亲和接口与 SIMD Only 思路。

■ 动态编组：挂载 1 名编译器骨干 + 1 名 AI 专家，补充跨层优化判断。

■ 结果：延迟优化到 2.557 μ s，FLA 类场景最高加速比 12.15 $\times$ 。

ATM 机制

紧急攻坚不靠长期借调，而靠短期集中优势能力。

FRI

0人介入

XX 平台批跑达标率汇总

■ 触发：平台批跑结果更新，需要汇总新增达标算子和 Skill 命中率。

■ Agent 判断：标准化统计任务，验证链路和日志结构完整，无需挂载人。

■ 自主执行：自动分析本周新增达标算子，更新 Skill 有效命中率。

■ 结果：0.6 $\times$ 性能达标率从 65% 到 70%，继续推进到 80%+。

ATM 机制

低风险标准任务由 Agent 消化，让人力留给高风险判断。

ATM 不是“所有人都并行参与”，而是“Agent 在正确时间调用正确能力”
每次挂载都留下 Pattern、Skill、日志和命中率统计，下一次同类任务的人工依赖会进一步下降。

TAKEAWAY

组织结构的目标，是 让人和 Agent 各归其位

Human IO Loop 解决人机协作的治理边界；ATM 解决任务、人才、Agent 的最优配置；两者合在一起，推动团队从固定职能组织走向任务驱动的动态作战组织。

人的价值

从“多做一步”变成“判断哪一步最值得人做”。

Agent 的价值

从“辅助个人”变成“支撑团队连续执行”。

组织的价值

从“固定分工”变成“业态组织”。

BOUNDARY CONDITIONS

Human IO Loop + ATM 的边界在哪里？从 R 到 D

不是所有任务都适合马上进入组织化流水线。先判断它处在 Research 还是 Development，再决定用探索循环还是 HIO + ATM 作战阵型。

R → D 从不确定性探索，走向可验证、可复用、可规模化交付。

R

Research: 高自由度探索, Agentic Coding

没有稳定范式，验证资产不足，过早治理会拖慢试错。

快速试错型 Demo 与创新头脑风暴

01 全新 GC 算法、新型编译架构、前沿预研；标准化治理会增加冗余。

高度依赖隐性经验的存量业务

02 经验只在专家脑中，没有文档、规则、Skills 和能力标签；Agent 难以自主适配。

无法搭建自动化验证体系的依赖型业务

03 强依赖线下环境、专属硬件、人工联合调试；缺少 Agent 自主迭代的可信反馈。

判定路径

Research

先证明路线可能成立

Evidence

形成指标、样例、失败边界

Pattern

沉淀流程、门禁、能力标
签

Development

D

Development: 高度固定流程, HIOL + ATM

规则清楚，流程高度固定，结果能被自动验证。

标准化、高重复度研发流程

01 标准库开发、常态功能迭代、社区问题批量处理；流程固定，验收标准清晰。

可分层校验、可自动修复的迭代业务

02 上游 bug 合入、LLVM 漏洞修复、兼容性迭代；测试、合规、性能门禁能跑起来。

长链路、多步骤、高复杂度系统任务

03 编译器核心特性、NPU 算子生态迁移、大型性能优化；需要多领域能力动态聚合。

准入四问

流程是否固定？

规则是否明确？

结果能否自动验?

经验能否沉淀复用?

OUTSIDE THE TWO PATTERNS · AGENT-AUGMENTED R&D

Research案例：GC 预研先验证路线，不急流水线化

Agent 先把方案、原型和数据跑出来；人再判断是否投入。

背景：CopyGC 在服务器端大内存场景下有改进空间

CopyGC 在小堆、小存活集下表现优异。但在服务器端高并发、大内存、长时运行场景下，存活对象变多，复制搬迁的账就会越来越重。

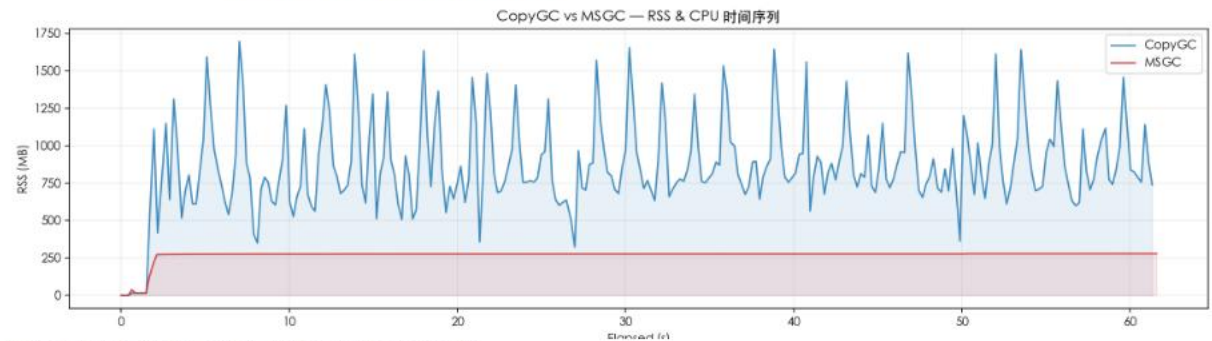
- 分配路径很快，核心是 t1Region 指针后移；回收时再把存活对象搬到一起。
- 大存活集下，搬迁成本上来以后，GC 时间变长，RSS 也更难压下去。

投入与效果对比

ESTIMATE → ACTUAL

研发投入	无AI评 AI实际		4人/120人月 1人/业余4周	60%探索
Peak RSS	Copy CMS		1696MB 280MB	-83.5%
GC Time	Copy CMS		98.6ms 8.3ms	-91.6%
TPS	Copy CMS		5066 5119	+1.0%

12GB 大内存场景：RSS 更低，CPU 只小幅上扬



NEWGC 的 RSS 曲线更低、更稳定，同时 CPU 平均占用只小幅上扬。

机制对照：搬迁压缩 vs. 原位清扫复用

CopyGC

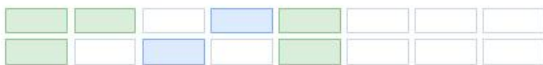
COPY COMPACT



优点是整理碎片很直接；代价是活对象越多，复制工作越重。

NEWGC

SIZE CLASS SWEEP



对象按大小落到 region，靠 Central Pool、lazy sweep 和 empty region 回收来复用内存。

Agent 的作用

细化
方案

编写
原型

构建
用例

性能
评估

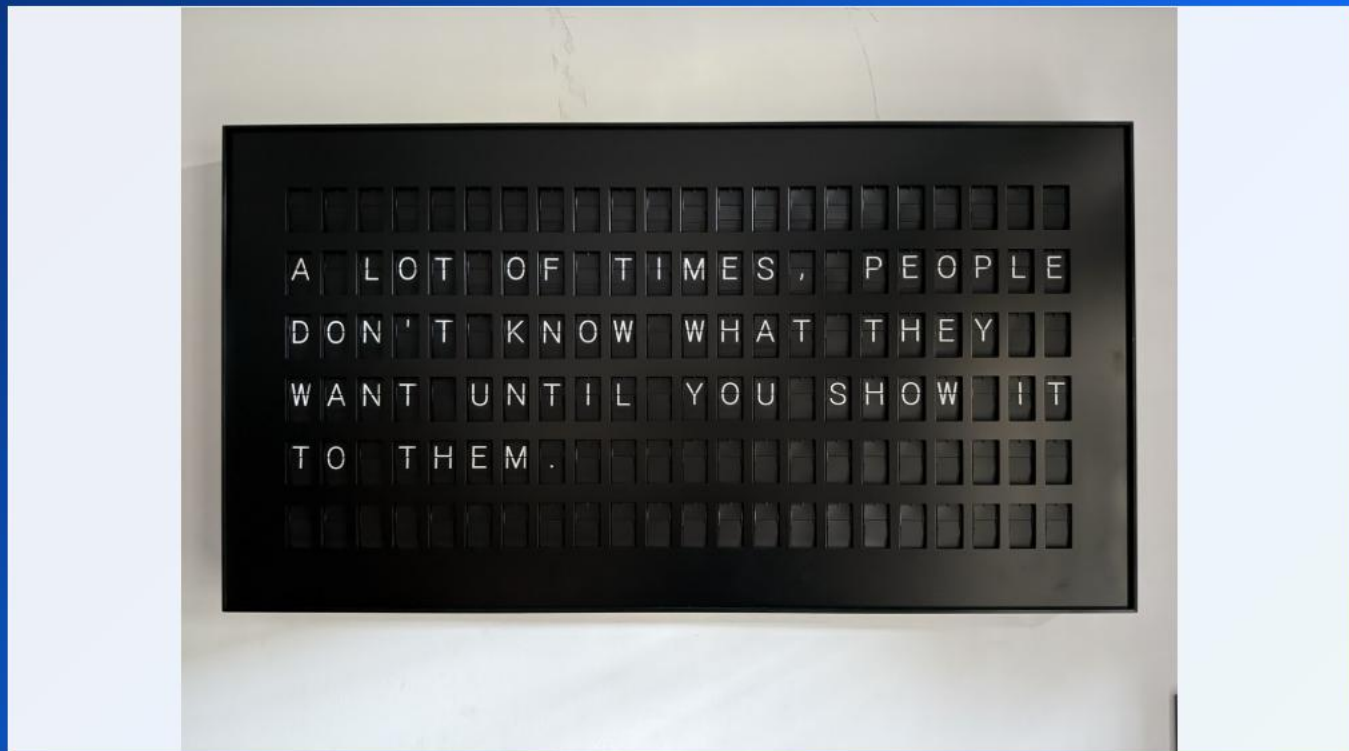
复盘
沉淀

BELIEF BEFORE EVIDENCE

相信 > 看见

AI 研发实践常常不是“看见了才相信”，而是先相信可行，再设计可验证路径，让结果出现。

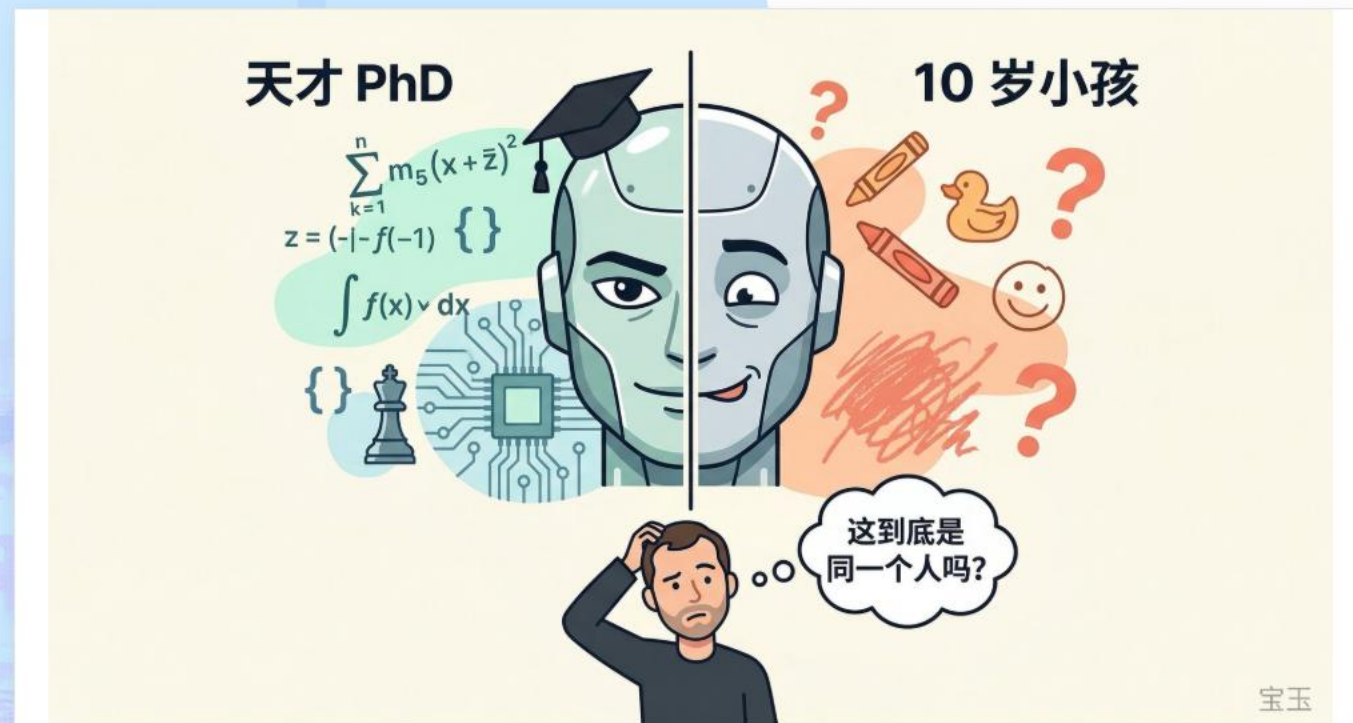
相信
>
看见



VERIFIABILITY IS THE BOUNDARY

Agent 只能改进可验证的东西

Agent 有远超人类的参差性，这是强化学习的影子；所以它只能稳定改进那些能被验证的东西。



V

把目标变成测试、基准、日志、回归、审查标准。没有验证，Agent 只能放大不确定性。

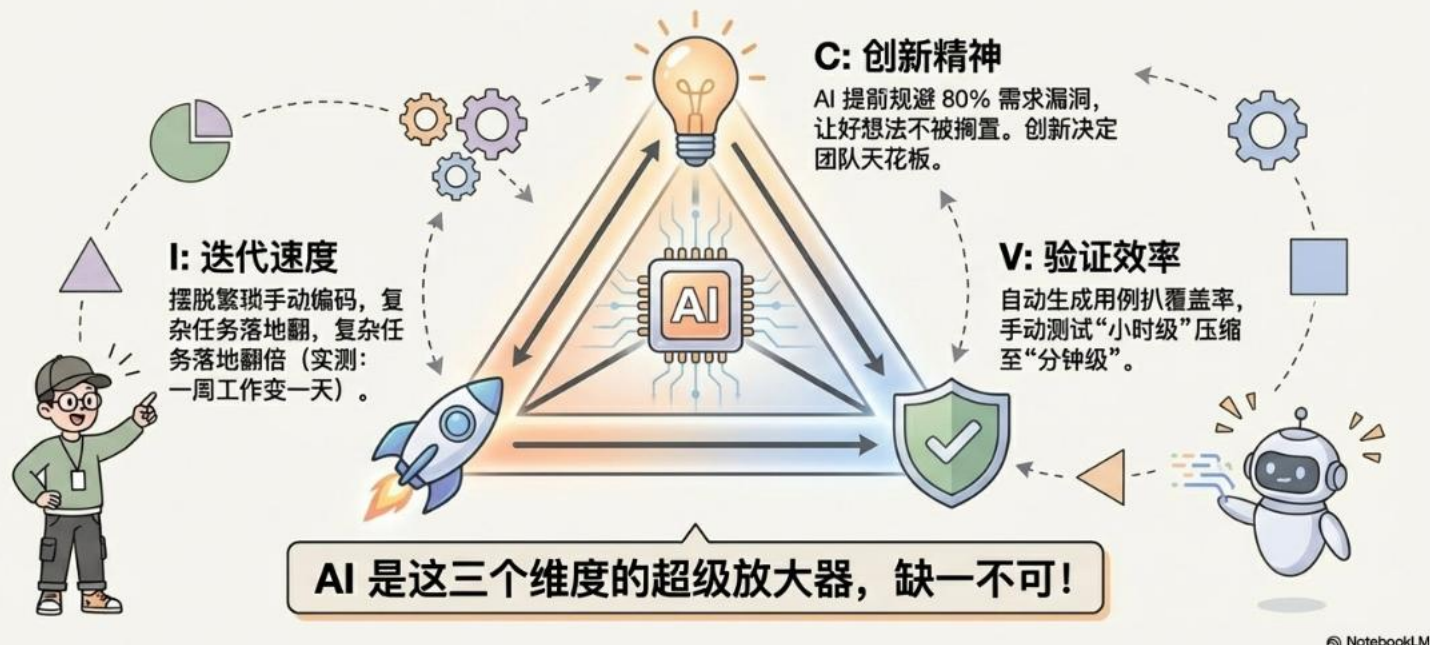
NEW PRODUCTIVITY FORMULA

价值重构：AI 驱动的研发效能新公式

效能不是单点效率，而是创新精神、迭代速度、验证效率三者相乘。

价值重构：AI 驱动的研发效能新公式

ACE 效能 = 创新精神 (C) × 迭代速度 (I) × 验证效率 (V)



极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 8月 📍 深圳 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海 🧑 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

AiCon

全球人工智能开发与应用大会

深圳站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 8 月 21-22 日 / 深圳湾万丽酒店



参会咨询

专题: Harness Engineering: 模型之外的智能体工程

专题出品人: 柴思远

智谱 / 副总裁



专题: 超级个体与蜂群智能的共生进化

专题出品人: 张昊阳

EvoMap / 创始人



专题: AI Agent 高价值商业场景实战

专题出品人: 李响 博士

平安科技 / 智能两核 AI 部总经理



专题: 超级 App 的 AI 原生化: 端智能、端云协同与用户体验重构

专题出品人: 黎三平

京东 / 端技术负责人



专题: AI 原生新范式: Coding Agent 重构软件研发全流程

专题出品人: 林香鑫

网易游戏 / 高级技术经理



专题: 智能体时代的数据供给与治理

专题出品人: 陈廷梁 (王赟)

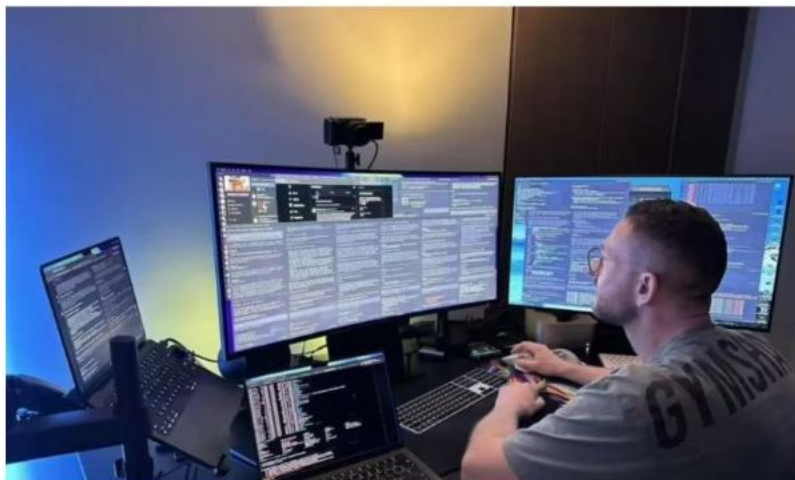
数新智能 / 创始人兼 CEO



SUPER INDIVIDUAL

AI 时代的超级个体

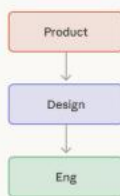
执行力可被无限供给时，判断力才是真正稀缺资源。



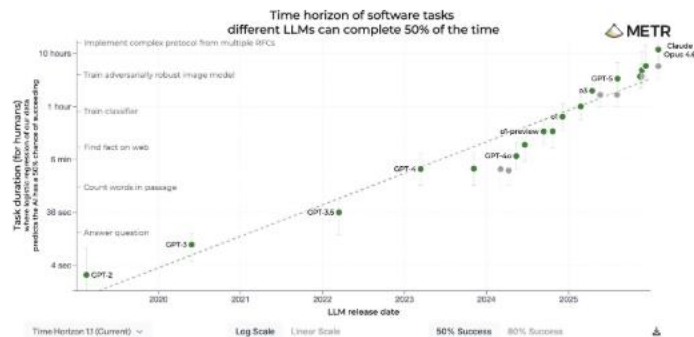
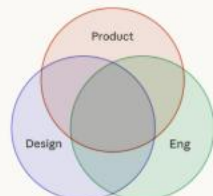
判断力锻造超级个体

The PM playbook on the AI exponential

Before AI



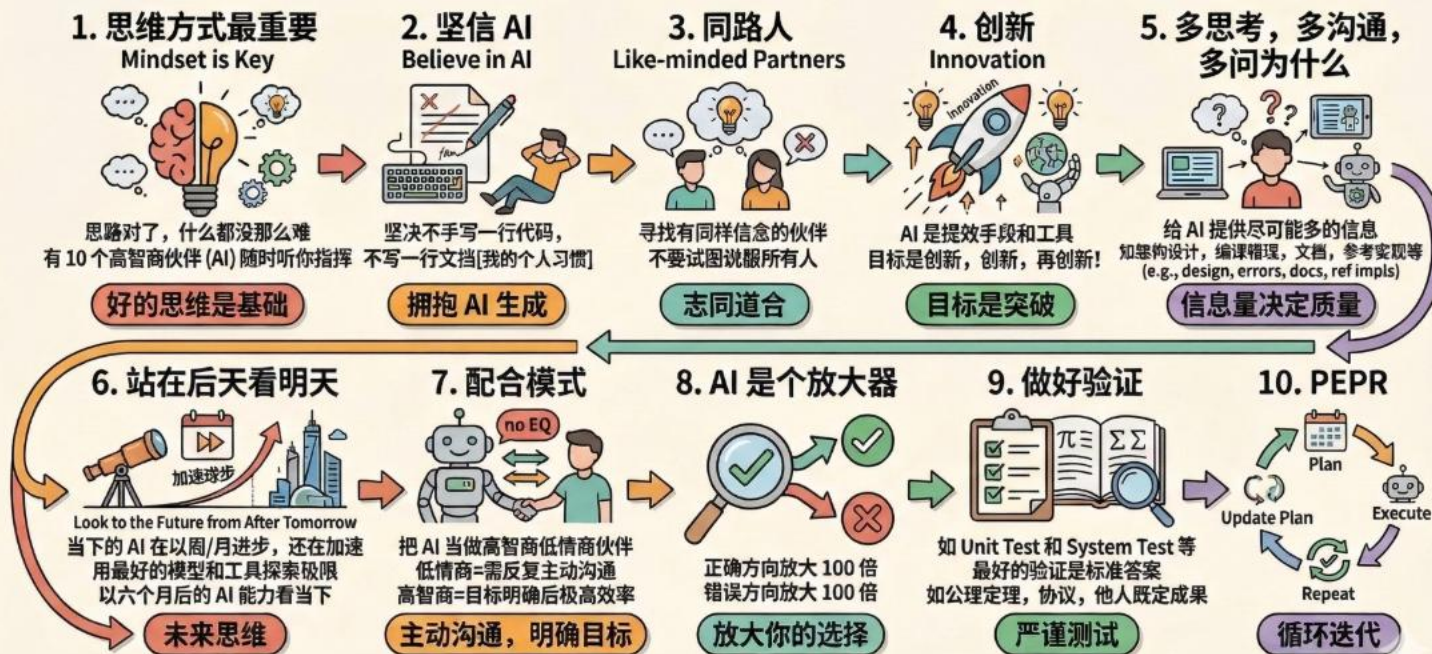
With AI



我的AI十条 · MAP 10

把个人经验沉淀为可复用原则：相信、验证、上下文、自动化、复盘，最终形成团队方法论。

我的AI十条 - MAP 10 (10 of My AI Principles)



THANK YOU

